Supplementary Material: Communication Costs and Verification of a Step-Controlled FPGA CGRA

Mattia Fait
University of Trento
Trento, Italy

***Abstract*—This supplement collects material that supports the paper but is not needed to follow its argument: controller and wire-level details, the host software stack, an illustration of capture spacing, alternative presentations of data already tabulated in the paper, the planned board-acceptance procedure with its (empty) report figure, and the chronology of the formal and implementation runs, including failed attempts. References of the form "Section P-x" point to the paper.**

S1. Controller and wire-level details

A. Controller state machine

Fig. S1 shows the controller of `cgra_ctrl` with the RTL state names. From `S_IDLE` the first received byte selects a branch. Short transactions (`ID`, `RST` and the rejection of an unknown opcode) reply at once through the shared transmit handshake. Bulk transfers accumulate their payload, receive and verify the checksum in a dedicated state and reply `ACK` or `NACK`. `RUN` produces N enabled PE captures at the configured cadence before acknowledging. The `EXEC` header state falls through into the same receive state as `WR` and the same step engine as `RUN`; only the return targets, held in two registers, differ. A watchdog guards every state that waits for a payload or checksum byte. After an uncertain transfer the library clears its cached identity and fails later calls with `CGRA_ERR_DESYNC`; it sends no implicit `RST`, because a controller waiting for a `RUN` argument would interpret that byte (`0x06`) as a step count.

B. A transaction on the wire

Writing west $= (1,0,0,0)$ and north $= (5,0,0,0)$ on a 4×4 device sends the command byte `0x03`, the four west and four north words little-endian, and the additive checksum:

```
03  0100 0000 0000 0000  0500 0000 0000 0000  06
```

The sixteen payload bytes sum to 6; the device replies `ACK` (`0x79`) or, on a mismatch, `NACK` (`0x1F`). Words are committed as they arrive, so a `NACK` does not restore the previous inputs.

S2. Host software stack

The C library owns transport and protocol: identification, configuration, input transfer, stepping and read-back primitives, checksums, the retransmission rule of Section P-III-C, and session shutdown after uncertain transfers. It caches the protocol version and geometry from one identification exchange and selects fused or unfused execution per chunk, so one binary drives either protocol version and every supported geometry. The in-process device model speaks the same byte protocols; it can also be exposed on a POSIX pseudo-terminal [1], so that the real serial code path is exercised without hardware. A command-line tool compiles a declarative configuration language into per-PE words and drives the library (Fig. S2). The public header defines buffer capacities, ownership and aliasing; a failure after computation has started can leave partial output, and calls sharing a handle or a physical device require caller synchronisation.

S3. Additional views of reported data

Fig. S3 illustrates the capture spacing that the multicycle argument of Section P-VI-D relies on, for $D = 3$ after a fused reset. Figs. S4 and S5 plot data that the paper reports in Table P-IX and Table P-VI. Fig. S6 shows an earlier, separately archived sweep of repeated dot products ($n = 16$–1024, $r = 1$–64, both vectors reused), whose counts agree with the closed forms checked by its own script; the paper's Fig. P-9 supersedes it by separating batching from reuse.

S4. Planned board acceptance

No board was available. The acceptance procedure is fixed in advance, so that a later run tests stated criteria rather than criteria chosen after the fact.

1) *Artifact.* Program only a bitstream whose run manifest passes `make check-artifact`.
2) *Link.* `cgra probe` must receive an identification reply whose version and geometry equal the elaborated generics, then pass its known-answer tests.
3) *Function.* The kernels of the paper must reproduce the scalar references on the board for both protocol versions and the resident path.
4) *Timing.* Measure per-byte time and per-transaction turnaround τ on the actual USB–UART bridge with warm-up and repeated runs, report distributions, state the warm-up, repetition and outlier policy [2], and compare with a timed CPU baseline.
5) *Recovery.* Interrupt transfers, assert reset and let the watchdog expire; the host must report the session as uncertain and recover only through the documented reset path.

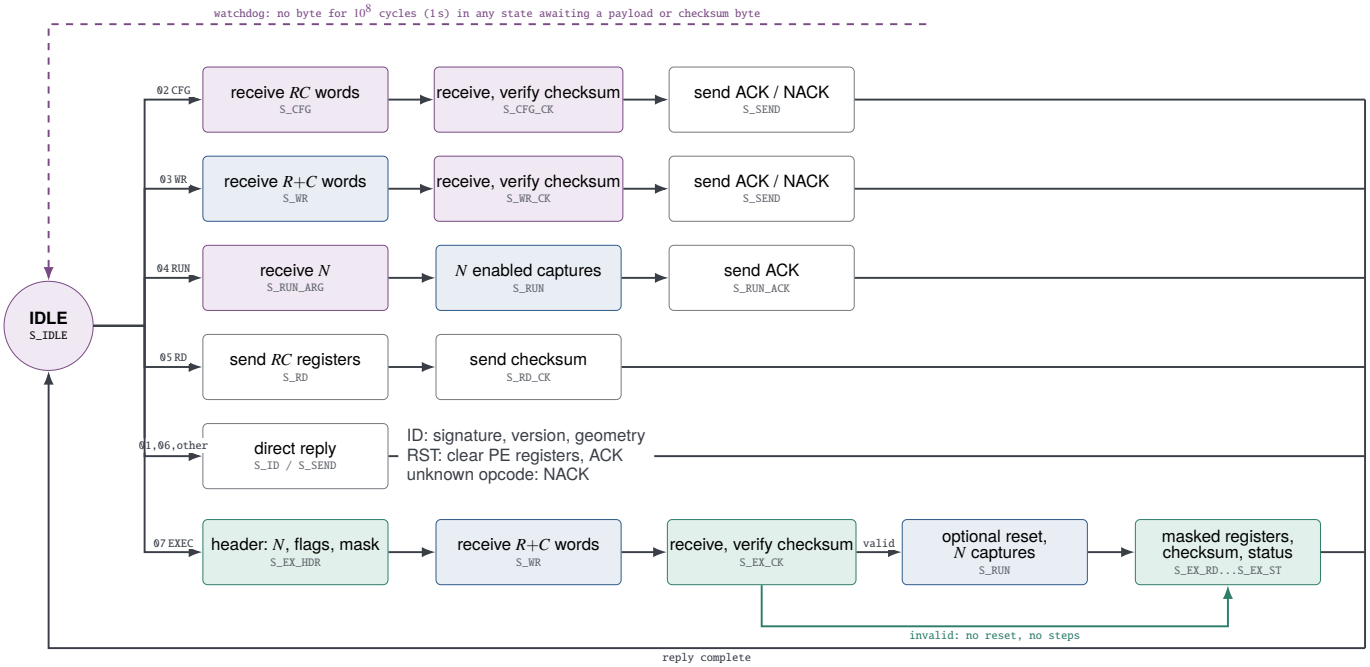


Fig. S1. Controller state machine of `cgra_ctrl`. Boxes give the function and the RTL state(s) in typewriter; byte loops and the shared transmit handshake are folded into the boxes. Blue states are shared by two commands (`WR`/`EXEC` reception, `RUN`/`EXEC` stepping) and return through a stored target; green states exist only for `EXEC`; white states transmit. A rejected `EXEC` checksum skips reset and stepping but still returns the current registers with `NACK`. Every reply ends in `S_IDLE`; the watchdog (dashed) aborts stalled reception.

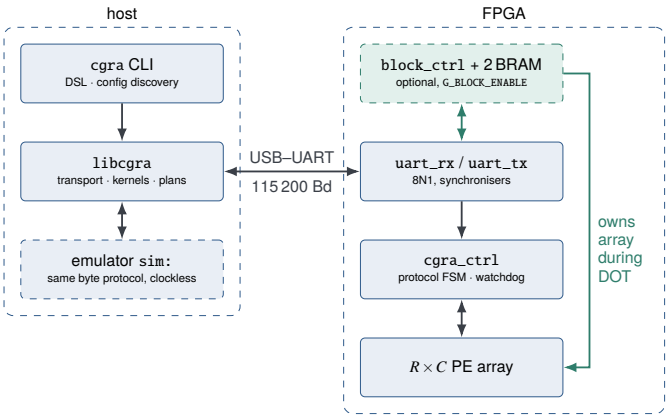


Fig. S2. Host/device architecture. The command-line tool compiles a declarative configuration language into per-PE words and drives the library, which reaches either the FPGA over a USB–UART bridge or the clockless emulator speaking the same byte protocol. The block controller (dashed, green) exists only when the extension is elaborated.

Fig. S7 shows the intended report. Its measured panels are empty: they are generated from `doc/data/board/`, which holds only column headers, and are marked NOT RUN. Panel (b) already shows the model of Eq. P-2 for $\tau = 0$ and $\tau = 16$ ms for later comparison. No synthetic data are shown as measurements.

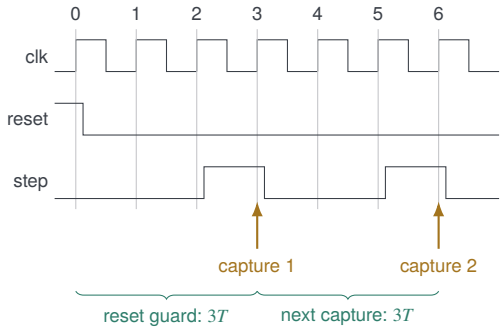


Fig. S3. Example with $D = 3$ after EXEC reset. Reset changes the PE registers at edge 0; the first step captures at edge 3, then at edge 6. Stable source registers and the initial guard are required as well as periodic spacing.

S5. Chronology of formal and implementation runs

Every run below is archived with its inputs, commands, tool versions, logs and source hashes; failed and inconclusive runs are kept as such.

Implementation. An initial synthesis and STA run on 23 September 2026 concerned an earlier design and is superseded. On 5 October a complete bitstream flow for the legacy 4×4 design passed all nine gates (WNS $+1.862$ ns, WHS $+0.098$ ns) and five 16-PE shapes were implemented from identical sources. The first period search failed its final gate, because the requested period (5.859375 ns) differed from the period applied by the tool (5.859 ns); after quantising periods to 1 ps and checking

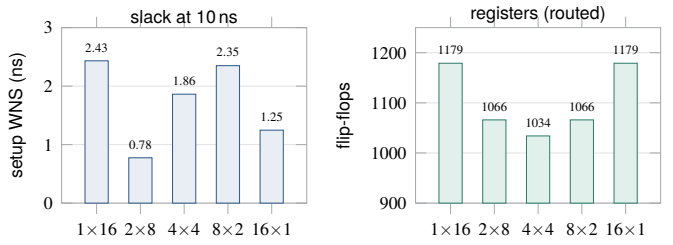


Fig. S4. Five 16-PE geometries implemented from identical sources. All meet the 10 ns constraint (positive slack, left). Register count follows the edge length $R+C$ (right; axis truncated at 900). Each point is one placement, so slack differences include placer variation.

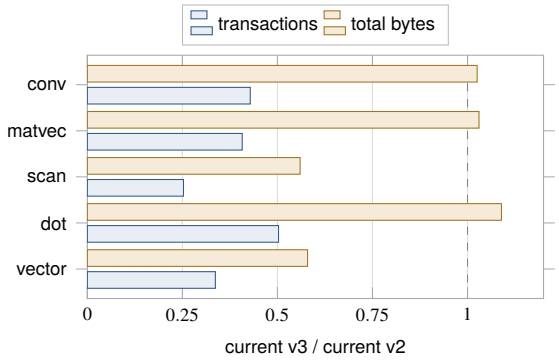


Fig. S5. Ratios of v3 to v2 transactions and bytes for the workloads of Table P-VI; values below one indicate a reduction. Counts, not times.

the applied clock, a second search on 6 October produced the 5.859 ns candidate. These runs used a top-level wrapper that predates the block extension. On 7 October the enabled block build first failed to elaborate (a boolean generic), then failed routed setup at -3.580 ns and, after adding operand registers, at -2.867 ns on a path from state decoding into the array multiplexers; an explicit ownership register with a proved guard led to the passing run of 8 October (WNS $+0.427$ ns). The synthesis pair of Table P-VII was produced the same day. For this paper the legacy 4×4 design was re-implemented on the current revision at 10 ns, reproducing the 5 October metrics exactly, and at 5.859 ns (WNS $+0.177$ ns, WHS $+0.122$ ns, identical to the archived candidate).

Formal. A monolithic 2×3 array proof timed out at 90 s and a first hierarchical PE formulation at 120 s; the array is therefore proved by composition with the separately proved PE transition. On 8 October the full suite then in place reported 27 PROVED tasks and 8 required failures. On 9 October an extended suite with the block properties reported 37 PROVED, 10 required failures and one UNKNOWN (block controller, $D=4$), where induction did not close at the chosen depth; a separate run PROVED that task after a reachable-state invariant was added as an assertion. Several cover attempts for the block controller timed out or hit frontend errors before the transcript driver was used. For this paper the complete full and cover suites were re-executed on the current sources: 38 PROVED tasks, all 10 negative and mutation tasks failing as required, no

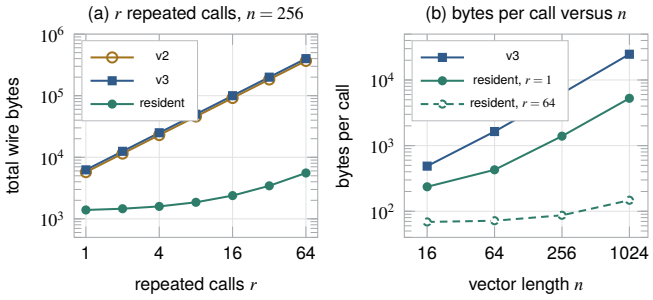


Fig. S6. Repeated dot products through the public API, 4×4 emulator, every result checked against a scalar reference. (a) Legacy traffic grows linearly with the number of calls r; resident traffic is dominated by the one-time upload. (b) Per-call bytes; for $r=64$ the upload is amortised. Counts, not times.

UNKNOWN, and 18 reached covers. Only this last execution is reported as the formal result of the paper.

References

[1] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013. [Online]. Available: https://www.informit.com/store/advanced-programming-in-the-unix-environment-9780321638007

[2] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2015, pp. 73:1–73:12. [Online]. Available: https://doi.org/10.1145/2807591.2807644

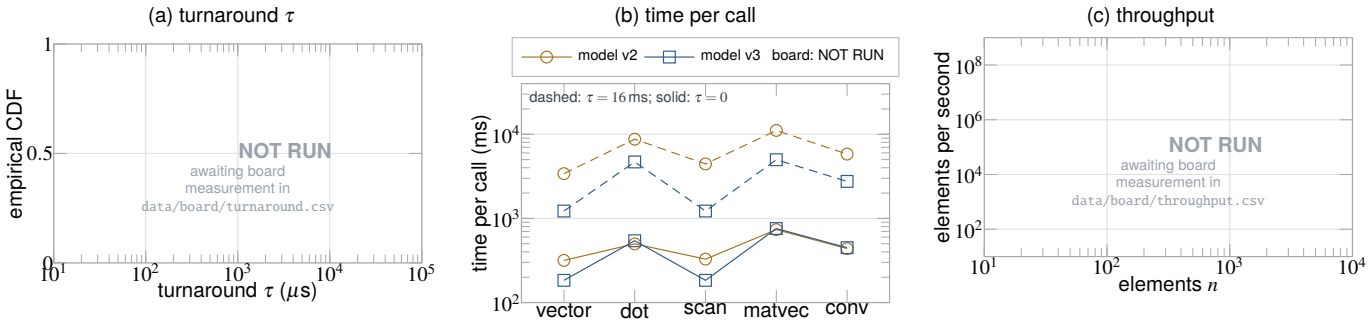


Fig. S7. Report format for board measurements: (a) empirical distribution of τ; (b) measured call time against the model (solid: $\tau = 0$; dashed: $\tau = 16\,\text{ms}$); (c) throughput against a timed CPU baseline. All measured series are NOT RUN.